

AI Agents & Agentic AI

From a single tool-using agent to systems that plan, act, and work together toward a goal.

Sourav Karmakar

souravkarmakar29@gmail.com

What we'll cover



01



Foundations

How agents work: the loop, autonomy levels, and ReAct.

02



What Makes an Agent

Tools & function-calling, memory, planning, and reflection.

03



Agentic Systems

Single- vs. multi-agent, coordination patterns, and where RAG fits.

04



Building & Frameworks

The build loop, the framework landscape, and a full example.

05



Reality & Responsible Use

Failure modes, evaluation, cost, and safety guardrails.

What is an AI agent?



An AI agent is an LLM that can decide and act in a loop: given a goal, it reasons about what to do, uses tools to act, observes the result, and repeats until the goal is met.



“AI agent”

The unit: one model running the reason–act loop with tools and memory to complete a task.



“Agentic AI”

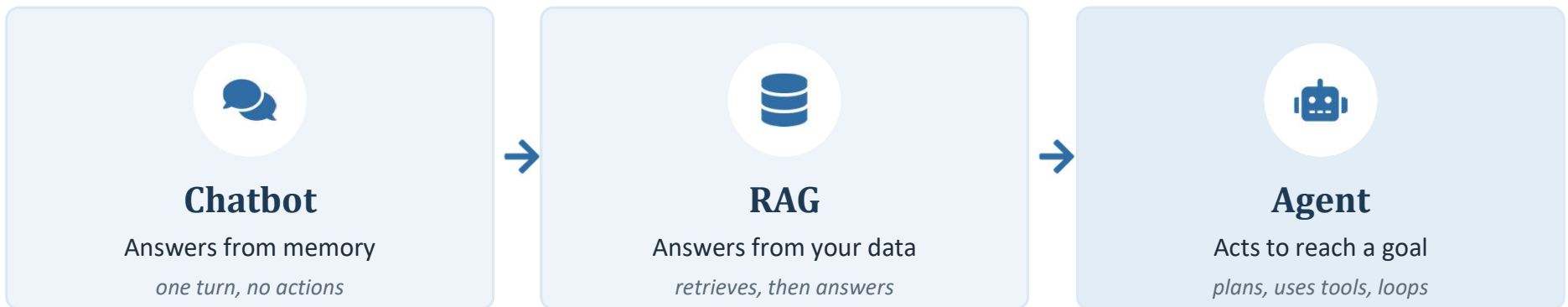
The paradigm: building systems that are goal-directed and multi-step — often several agents working together.

THE BIG PICTURE

From chatbot → RAG → agent



Each step gives the model more ability to get things done. Agents add the power to act, not just answer.



Running example: a research-assistant agent. Ask it to “research topic X and write a short report.” It will plan, search the web, read pages, and write up findings with citations.

PART

01



Foundations: How Agents Work

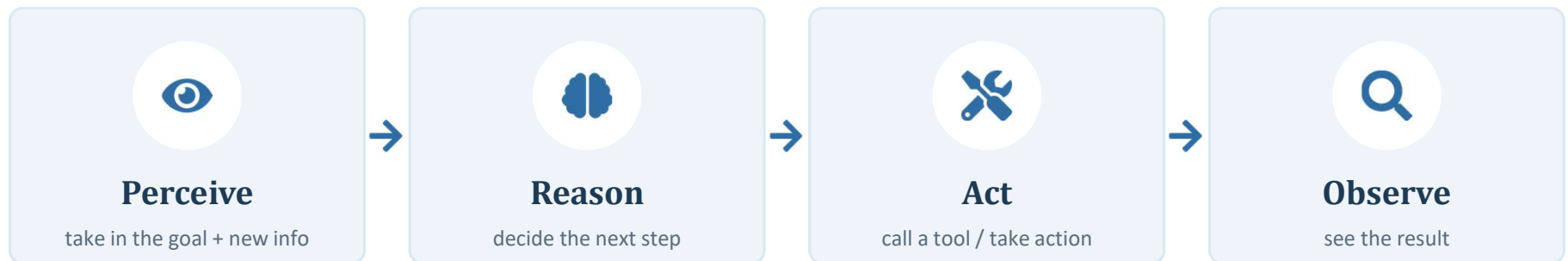
The single idea everything rests on — an agent is a loop. Plus how it differs from a chatbot and from RAG.

FOUNDATIONS

The agent loop



An agent runs the same cycle over and over until the task is done. Each pass moves it closer to the goal.



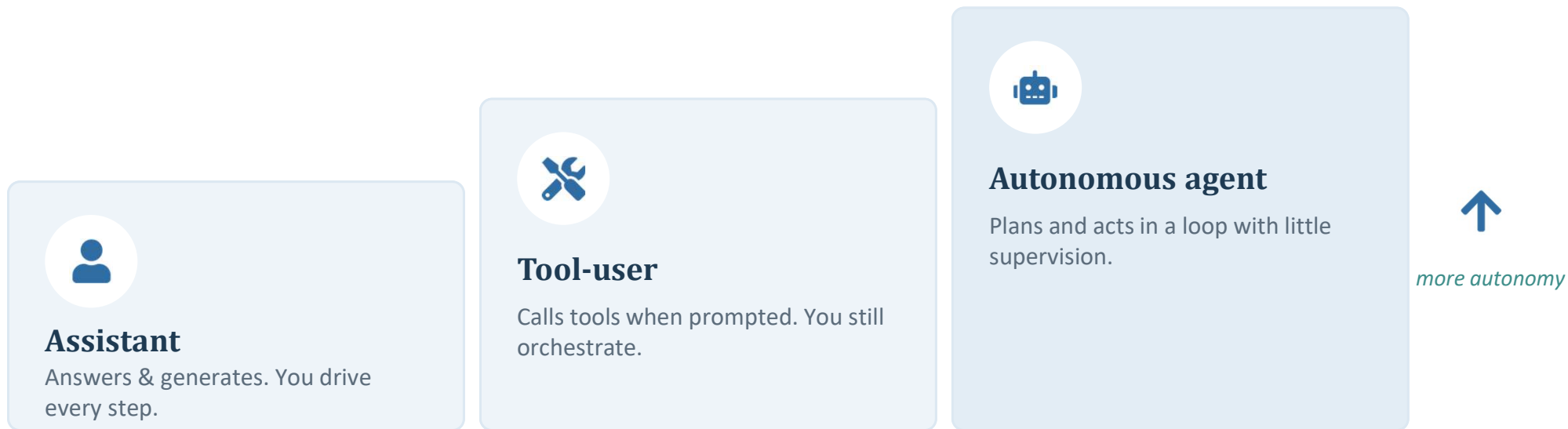
repeat until the goal is met (or a limit is hit)

FOUNDATIONS

Levels of autonomy



“Agent” is a spectrum, not a switch. Capability rises as the human hands over more of the decision-making.



Agent vs. chatbot vs. RAG



All three use an LLM. The difference is what they can do with it.

	 Chatbot	 RAG	 Agent
Source of answer	Training memory	Your documents	Tools + data + memory
Can take actions?	No	No (just retrieves)	Yes — that's the point
Steps per task	One	One (retrieve+answer)	Many, in a loop
Best for	Q&A, drafting	Grounded Q&A	Multi-step goals

FOUNDATIONS

ReAct: reason + act, interleaved



The most common agent pattern. The model alternates a private Thought with an Action, reads the Observation, and continues — thinking out loud between steps.

RESEARCH ASSISTANT · TRACE

Thought: I need recent sources on the topic.
Action: `web_search("topic X 2025 overview")`
Observation: 3 results returned.
Thought: The 2nd looks authoritative — read it.
Action: `read_page(url)`
Observation: Key facts extracted.
Answer: Draft the report with citations.

Why ReAct works

- Thinking first → better action choices
- Acting → brings in real, fresh facts
- The trace is readable & debuggable
- Foundation under most agent frameworks

PART

02



What Makes an Agent

Four ingredients turn an LLM into an agent: tools, memory, planning, and reflection.

INGREDIENTS

Tools & function-calling



Tools give the model hands. Function-calling lets it say “run this tool with these inputs” — your code runs it and returns the result.



Typical tools



Web search



Retrieval (RAG)



Calculator



Run code



Read / write files



Call an API

INGREDIENTS

What a tool definition looks like



You describe each tool to the model: a name, what it does, and its inputs. Good descriptions = the model picks the right tool at the right time.

TOOL DEFINITION (SCHEMA)

```
{
  "name": "web_search",
  "description": "Search the web and
    return the top results for a query.",
  "parameters": {
    "query": { "type": "string" }
  }
}
```

MODEL EMITS A CALL

```
web_search({
  "query": "topic X 2025"
})
```

TOOL RETURNS

```
[ {title, url, snippet},
  {title, url, snippet},
  ... ]
```

Memory: short-term vs. long-term



Agents need to remember — both within a task and across tasks. These are two different mechanisms.



Short-term

The current task

- Lives in the context window
- Holds the goal, recent steps & observations
- Limited & temporary — lost when it overflows



Long-term

Across tasks & sessions

- Stored outside the model (often a vector DB)
- Retrieved when relevant — just like RAG
- Remembers facts, preferences, past results

INGREDIENTS

Planning & task decomposition



For anything non-trivial, the agent first breaks the goal into steps, then works through them. Planning prevents aimless wandering.

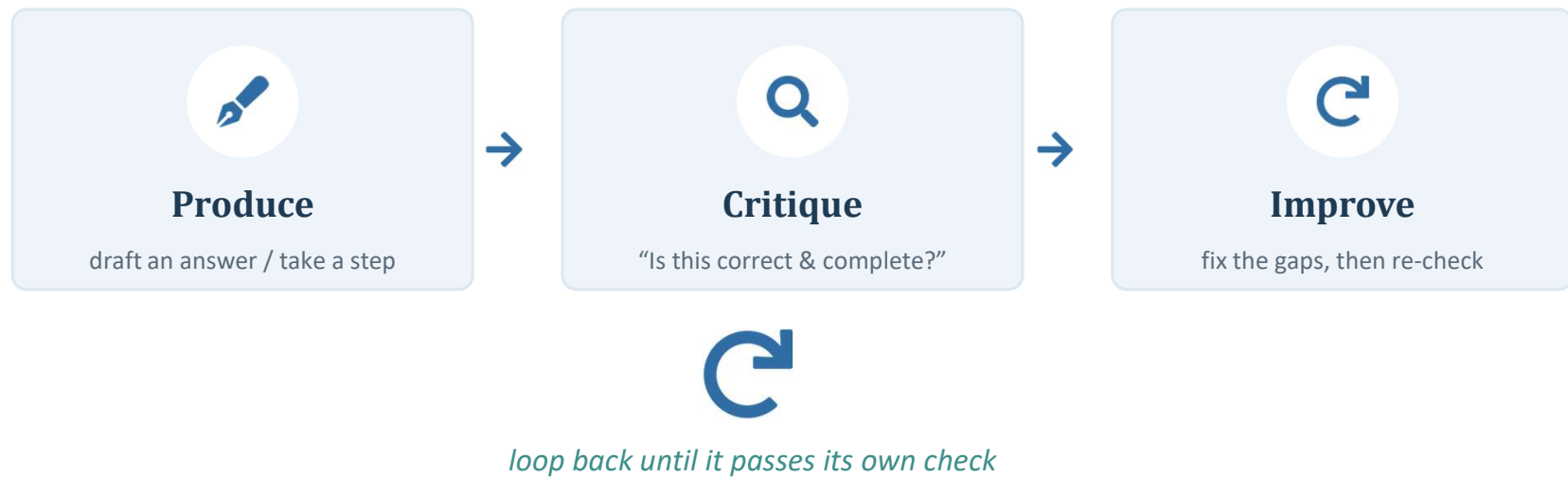


INGREDIENTS

Reflection & self-correction



Strong agents check their own work. After acting, the agent critiques the result and tries again if it falls short — catching errors before you do.



PART

03



Agentic Systems: Many Agents

One agent is powerful; several specialized agents working together can tackle bigger, messier goals.

Single-agent vs. multi-agent



Instead of one agent doing everything, split the work across specialists — each with its own role, tools, and prompt.

Single agent



One agent handles planning, searching, and writing by itself.

Simple · cheaper · can overload on big tasks

Multi-agent



Planner



Researcher



Writer

Specialists coordinate — each does one job well.

Scales to complex work · more moving parts & cost

Common multi-agent patterns



A few reusable shapes cover most multi-agent designs:

Planner – Executor



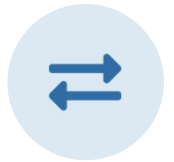
Orchestrator – Workers



Evaluator – Optimizer



How agents coordinate



Multiple agents only help if they share information well. Coordination is mostly about passing the right messages and results.



Messages

Agents send each other text — tasks, results, questions — like a team chat.



Shared memory

A common scratchpad / store all agents can read and write.



Handoffs

One agent passes control (and context) to another better suited.



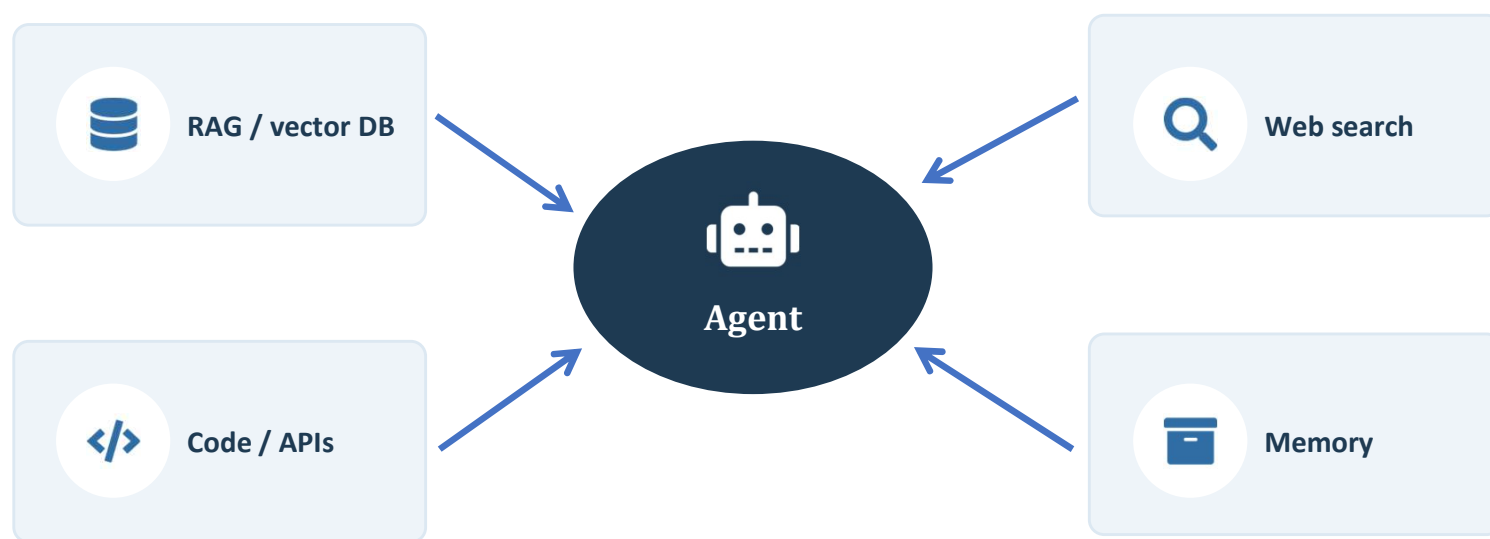
A manager

An orchestrator routes work and decides who does what next.

Where RAG & tools fit in



An agentic system ties everything together: the agent is the brain; RAG and tools are capabilities it reaches for as needed.



RAG becomes just one tool the agent can call — “look it up in our docs.”

PART

04



Building & Frameworks

How to actually assemble an agent, the tools that help, and the running example end to end.

BUILDING

The build loop



Building an agent is itself iterative. Define what it's for, give it tools, wrap it in the loop, add guardrails — then test and refine.



Start with the simplest thing that works — one agent, few tools — then add complexity only when testing demands it.

BUILDING

The framework landscape



You can build the loop by hand, but frameworks handle the plumbing — tool-calling, memory, and multi-agent coordination. Awareness level:



LangGraph

Graph-based control of agent steps & state. Fine-grained and flexible.



CrewAI

Role-based “crews” of agents with simple, readable setup.



AutoGen

Conversation-driven multi-agent systems (from Microsoft).



OpenAI Agents SDK

Lightweight agents, tools & handoffs from OpenAI.

Pick by fit, not hype — and you can always start framework-free to learn the moving parts.

BUILDING

End-to-end: the research-assistant agent



All the pieces in one run — goal in, cited report out.

	Goal	“Research topic X and write a short, cited report.”
	Plan	Break into: search → read → compare → write → cite
	Act (tools)	web_search(...), then read_page(...) on the best hits
	Observe + remember	Extract key facts; keep sources in memory
	Reflect	Every claim cited? Gaps? → search again if needed
	Deliver	Short report with inline citations to sources

PART

05



Reality & Responsible Use

Autonomy cuts both ways. Know how agents fail, what they cost, and how to keep them safe.

Failure modes



More autonomy means more ways to go wrong — and small errors can compound across steps.



Getting stuck in loops

Repeating the same action without progress — burning time and tokens.



Runaway cost

Each step is more model calls; an unbounded agent can rack up a huge bill.



Compounding errors

An early wrong step misleads every step after it.



Bad / unsafe tool calls

Wrong arguments, or invoking a powerful tool it shouldn't have.

Fixes preview: step limits, budgets, validation, and human checks — next slides.

Evaluating agents



Judge more than the final answer. With agents, how they got there matters too.



Outcome

Did it achieve the goal?

- Task success rate on real cases
- Answer correct & complete
- Sources valid (for the report)



Process

How did it get there?

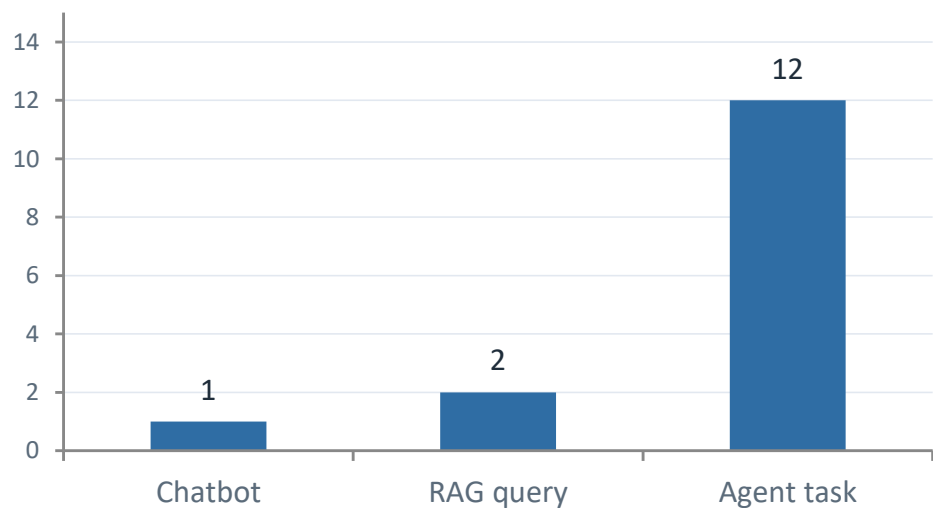
- Steps taken (fewer is better)
- Right tools, right arguments?
- Cost & time per task

REALITY

Cost & latency



Agents are multi-step, so they make many model calls per task. That's the price of autonomy — manage it deliberately.



Illustrative: a single agent task can be 10×+ the calls of one chatbot reply.

Keeping it in check

- Cap steps and set a budget per task
- Use a smaller model for easy sub-steps
- Cache and reuse results; avoid repeat work
- Stop early once the goal is met
- Prefer the simplest design that works

REALITY

Safety & guardrails



An agent that can act can act badly. Because it operates with autonomy, safety must be designed in — not bolted on.



Human-in-the-loop

Require approval before risky actions (sending, paying, deleting).



Least privilege

Give the smallest set of tools & permissions the task needs.



Limits & sandboxing

Cap steps/spend; run code & actions in a safe, isolated space.



Untrusted input

Treat web/tool content as untrusted — it can carry injected instructions.

WRAP-UP

Key takeaways



If you remember only these, you understand agents:

- ✓ An agent is an LLM in a loop: perceive → reason → act → observe, toward a goal.
- ✓ Four ingredients: tools, memory, planning, and reflection.
- ✓ Tools (function-calling) give the model the ability to act, not just answer.
- ✓ Agentic AI scales this to multiple specialized agents that coordinate.
- ✓ RAG and tools become capabilities the agent chooses to use.
- ✓ Autonomy raises cost and risk — evaluate, limit, and keep humans in the loop.

KEEP LEARNING

Where to go next



Build a single agent first

One model, the loop, two tools (search + read). Add more only when needed.



Try a framework

LangGraph, CrewAI, AutoGen, or the OpenAI Agents SDK once you grasp the loop.



Add guardrails early

Step limits, budgets, and a human check on risky actions from day one.



Evaluate on real tasks

Track success rate, steps, and cost as you iterate.

Beyond this course (not covered): RL-trained agents · deep orchestration/graph internals · computer-use & browser agents · Model Context Protocol (MCP) · formal multi-agent negotiation · production agent infrastructure.

Thank You